

Laboratorio di Calcolo

A.A. 2026/2027, Canale A-Dec

Giacomo Artoni, Lorenzo Caprini

Algoritmi e selezione

Il flusso di esecuzione

Finora le istruzioni venivano eseguite **una dopo l'altra**, sempre tutte, sempre nello stesso ordine.

- Gli algoritmi della lezione 7 però contenevano **decisioni** ("se $b = 0$…") e **ripetizioni** ("torna al passo 1").
- Teorema di **Böhm–Jacopini** (1966): ogni algoritmo si può scrivere con tre sole strutture di controllo.
- Usare solo queste tre strutture si chiama **programmazione strutturata**: il codice si legge dall'alto in basso, senza "salti".

Struttura	Significato	In C
Sequenza	un'istruzione dopo l'altra	`;` e blocchi `{ }`
Selezione	scegliere **se** eseguire	`if` , `else`
Iterazione	**ripetere**	`while` , `for`

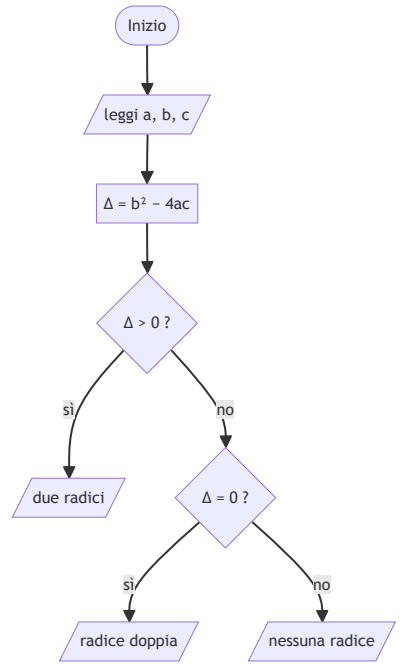
Oggi: la **selezione**. Nella prossima lezione: l'**iterazione**.

Diagrammi di flusso

Un **diagramma di flusso** (*flowchart*) rappresenta un algoritmo con blocchi e frecce.

Forma	Significato
ovale	inizio / fine
parallelogramma	ingresso / uscita dati
rettangolo	un'operazione
rombo	una **decisione** (sì / no)

- Esempio: le radici di $ax^2 + bx + c = 0$.
- Dal rombo escono **due** frecce: il programma prende strade diverse a seconda dei dati.
- Utile per progettare **prima** di scrivere il codice (fase 2 della lezione 7).



Operatori relazionali

Confrontano due valori. Il risultato è un **int**: `1` se il confronto è vero, `0` se è falso.

Operatore	Significato	`3 ? 5`
`==`	uguale	`0`
`!=`	diverso	`1`
`<`	minore	`1`
`>`	maggiore	`0`
`<=`	minore o uguale	`1`
`>=`	maggiore o uguale	`0`

```c
int a = 3, b = 5;
printf("%d\n", a < b);
printf("%d\n", a == b);
printf("%d\n", a != b);
```

```
1
0
1
```

- In C non esiste un tipo "booleano" di base: **vero** e **falso** sono numeri interi.
- Qualunque valore **diverso da zero** è considerato vero; solo `0` è falso.
- Uguaglianza: `==`, con **due** segni di uguale. Un solo `=` è l'assegnamento!

Operatori logici

Combinano più condizioni.

`p`	`q`	`p && q` (AND)	`p || q` (OR)	`!p` (NOT)
`0`	`0`	`0`	`0`	`1`
`0`	`1`	`0`	`1`	`1`
`1`	`0`	`0`	`1`	`0`
`1`	`1`	`1`	`1`	`0`

```
1   // l'angolo sta nel primo quadrante?
2   theta >= 0 && theta <= M_PI / 2
3
4   // la misura e' fuori dall'intervallo [a, b]?
5   x < a || x > b
```

Attenzione: niente catene di confronti

```
1   if (0 < x < 1)       // SBAGLIATO
2   if (0 < x && x < 1)  // corretto
```

```
1   warning: comparisons like ‘X<=Y<=Z’ do not have
2     their mathematical meaning [-Wparentheses]
```

- `0 < x < 1` viene letto come `(0 < x) < 1`: confronta con `1` un risultato che vale `0` o `1`.
- Con `x = 5`: `(0 < 5)` vale `1`, e `1 < 1` vale `0`. Con `x = -3`: `0 < 1` vale `1`, cioè **vero**!

Cortocircuito e precedenza

- `&&` e `||` valutano gli operandi **da sinistra a destra** e si fermano appena il risultato è noto.
- `p && q`: se `p` è falso, `q` **non viene calcolato** (il risultato è comunque `0`).
- `p || q`: se `p` è vero, `q` non viene calcolato.
- Si usa per **proteggere** un'operazione pericolosa:

```
1   // velocita' media sopra il limite?
2   if (t != 0 && d / t > v_max)
```

Se `t` vale zero, la divisione **non viene eseguita**.

Precedenza	Operatori
alta	`!` `-` (unari) `++` `--`
	`*` `/` `%`
	`+` `-`
	`<` `<=` `>` `>=`
	`==` `!=`
	`&&`
	`||`
bassa	`=` `+=` `-=` …

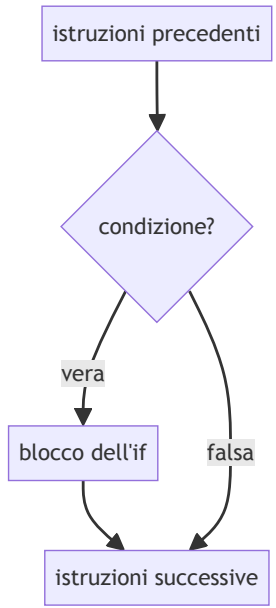
`&&` viene prima di `||`, come il prodotto prima della somma.
Nel dubbio: **parentesi**.

L’istruzione if

```c
if (condizione) {
    istruzioni;    // eseguite solo se
                   // la condizione e' vera
}
```

- La condizione va **tra parentesi tonde**.
- Se vale `0` (falso), il blocco viene **saltato** e si prosegue dopo la `}`.
- Se è diversa da `0` (vero), il blocco viene eseguito.

```c
if (v > c) {
    printf("Attenzione: v > c!\n");
}
```



Esempio: verificare l'input

Nella lezione 13 abbiamo visto che `scanf` restituisce il numero di valori letti: ora possiamo **controllarlo**.

```c
#include <stdio.h>

int main() {
    const double g = 9.81;   // [m/s^2]
    double m;                // [kg]

    printf("Massa [kg]: ");
    if (scanf("%lf", &m) != 1) {
        printf("Errore: non e' un numero\n");
        return 1;
    }
    if (m <= 0) {
        printf("Errore: la massa deve essere positiva\n");
        return 2;
    }
    printf("Peso: %g N\n", m * g);
    return 0;
}
```

```
$ ./massa
Massa [kg]: 2
Peso: 19.62 N
$ ./massa
Massa [kg]: -3
Errore: la massa deve essere positiva
$ echo $?
2
$ ./massa
Massa [kg]: kg
Errore: non e' un numero
```

- `return` dentro un `if` termina subito il programma.
- Un valore diverso da `0` segnala che qualcosa è andato storto (lezione 8).

if … else, blocchi e indentazione

```c
if (condizione) {
    // eseguito se vera
} else {
    // eseguito se falsa
}
```

- Esattamente **uno** dei due blocchi viene eseguito.
- Se un blocco contiene una sola istruzione, le graffe si possono omettere…
- …ma è più sicuro metterle **sempre**: per il C conta la `{ }`, non l'indentazione!

```c
if (delta < 0)
    printf("Nessuna radice reale\n");
    return 1;    // NON fa parte dell'if!
```

```
warning: this 'if' clause does not guard...
  [-Wmisleading-indentation]
note: ...this statement, but the latter is
  misleadingly indented as if it were guarded
  by the 'if'
```

Il `return 1;` viene eseguito **sempre**: il programma termina anche quando ci sono radici reali.

Più casi: else if

```c
#include <stdio.h>
#include <math.h>

int main() {
    double a, b, c;

    printf("Coefficienti a, b, c: ");
    scanf("%lf %lf %lf", &a, &b, &c);

    double delta = b * b - 4 * a * c;

    if (delta > 0) {
        double x1 = (-b - sqrt(delta)) / (2 * a);
        double x2 = (-b + sqrt(delta)) / (2 * a);
        printf("Due radici reali: %g e %g\n", x1, x2);
    } else if (delta == 0) {
        printf("Una radice doppia: %g\n", -b / (2 * a));
    } else {
        printf("Nessuna radice reale\n");
    }
    return 0;
}
```

```
$ ./secondo
Coefficienti a, b, c: 1 -3 2
Due radici reali: 1 e 2
$ ./secondo
Coefficienti a, b, c: 1 -2 1
Una radice doppia: 1
$ ./secondo
Coefficienti a, b, c: 1 0 1
Nessuna radice reale
```

- Si esegue solo il **primo** blocco con condizione vera; l'`else` finale raccoglie il resto.
- **Test:**
  $x^2 - 3x + 2 = (x - 1)(x - 2)$ ✓
- E se $a = 0$?
  Da **migliorare**!

Confrontare numeri reali

```c
double e = (0.4 - 0.1) / (0.3 - 0.0);

printf("e = %.17g\n", e);
printf("%d\n", e == 1.0);
printf("%d\n", fabs(e - 1.0) < 1e-9);
```

```
e = 1.0000000000000002
0
1
```

- Matematicamente $e = 0.3/0.3 = 1$, ma `0.4 - 0.1` non è esattamente `0.3` (lezione 4).
- `==` tra `double` risponde "no" per un errore di arrotondamento.

Si confronta con una tolleranza

```c
const double TOL = 1e-9;

if (fabs(a - b) < TOL) {
    // a e b sono "uguali"
}
```

- La tolleranza dipende dal problema: deve essere molto maggiore dell'errore di arrotondamento, ma molto minore della precisione che ci interessa.
- Per numeri molto grandi o molto piccoli conviene una tolleranza **relativa**: `fabs(a - b) < TOL * fabs(b)`.

Anche `delta == 0` nell'esempio precedente è un confronto rischioso!

Esempio: classificare un urto

Coefficiente di restituzione: $e = \dfrac{u_2 - u_1}{v_1 - v_2}$ (velocità v prima e u dopo l'urto).

```c
const double TOL = 1e-6;
double v1, v2, u1, u2;   // [m/s]

printf("v1 v2 prima, u1 u2 dopo [m/s]: ");
scanf("%lf %lf %lf %lf", &v1, &v2, &u1, &u2);

double e = (u2 - u1) / (v1 - v2);
if (fabs(e - 1) < TOL) {
    printf("Urto elastico\n");
} else if (fabs(e) < TOL) {
    printf("Urto completamente anelastico\n");
} else if (e > 0 && e < 1) {
    printf("Urto anelastico\n");
} else {
    printf("Dati non fisici!\n");
}
```

Input	Output
`2 0 0 2`	`Urto elastico`
`2 0 1 1`	`Urto completamente anelastico`
`2 0 0.5 1.5`	`Urto anelastico`
`0.3 0 0.1 0.4`	`Urto elastico`
`2 0 0 3`	`Dati non fisici!`

- Con `0.3 0 0.1 0.4` si ottiene `e = 1.0000000000000002` : senza tolleranza, "dati non fisici".
- E se `v1 == v2` ? Manca un controllo!

Le trappole di if

1. = al posto di ==

```c
if (n = 0) {
    printf("Nessuna misura\n");
}
```

```
warning: suggest parentheses around assignment
  used as truth value [-Wparentheses]
```

- `n = 0` **assegna** `0` a `n`, e vale `0`: la condizione è **sempre falsa**, e intanto `n` è stato cancellato!
- È un'espressione valida: senza `-Wall` nessun avviso.

2. Il punto e virgola di troppo

```c
if (m <= 0); {
    printf("Massa non valida\n");
}
```

```
warning: suggest braces around empty body in
  an ‘if’ statement [-Wempty-body]
```

- Il `;` è un'istruzione **vuota**: è lei il corpo dell'`if`.
- Il blocco successivo viene eseguito **sempre**.
- Dopo `if (...)` non va mai il punto e virgola.

if annidati e dangling else

```
if (v > 0)
    if (v > c)
        printf("Impossibile!\n");
else
    printf("Velocita' negativa\n");
```

```
warning: suggest explicit braces to avoid
  ambiguous ‘else’ [-Wdangling-else]
$ ./velocita
10
Velocita' negativa
```

- Un `if` può contenere un altro `if`: si dice **annidato**.
- Un `else` si lega sempre all'`if` **più vicino** ancora senza `else`, qualunque sia l'indentazione.
- Qui l'`else` appartiene a `if (v > c)`!

Con le graffe è tutto chiaro

```
if (v > 0) {
    if (v > c) {
        printf("Impossibile!\n");
    }
} else {
    printf("Velocita' negativa\n");
}
```

Regola d'oro: **graffe sempre**, anche per una sola istruzione. Costano due caratteri e risparmiano ore di debug.

L’operatore condizionale

Per scegliere tra **due valori** esiste un’espressione compatta: `cond ? se_vero : se_falso`

```c
double v = -3.5;
double modulo = (v >= 0) ? v : -v;
printf("%g\n", modulo);
```

```
3.5
```

```c
printf("%d misur%s\n", n,
       (n == 1) ? "a" : "e");
```

```
1 misura
4 misure
```

- È l’unico operatore del C con **tre** operandi (*ternario*).
- È un’**espressione**: produce un valore, che si può assegnare o passare a `printf`.
- Equivale a un `if ... else` che assegna la stessa variabile nei due rami.
- Usatelo solo per scelte **semplici**: per tutto il resto, `if`.